

scikit-verify: Extracting and Checking the Mathematics of Numerical Python

Aadya Chinubhai

AADYACHINUBHAI@GMAIL.COM

School of Engineering, Santa Clara University

Editor:

Abstract

scikit-verify is a Python library that recovers the mathematical expression a NumPy program computes and checks it against a specification. It runs the target function once on symbolic inputs and returns a SymPy expression together with the branch conditions under which that expression holds. Given a formula supplied by the user, it decides whether the code computes that formula, and it does so on every reachable branch by negating each recorded condition and querying the Z3 solver. Operations with no exact symbolic form are reported rather than approximated. The library needs no annotation of the target code and is available on PyPI under a BSD-3 license.

Keywords: software verification, symbolic execution, numerical computing, testing, computer algebra

1 Introduction

Errors in numerical software are often mathematical rather than behavioral. A function can execute without error, return values in the expected range, and still compute a different quantity than intended. Common cases include a sample variance that normalizes by n rather than $n - 1$, a data-dependent branch that is correct on the tested inputs but not on others, and a piecewise function assumed to be differentiable at a point where it is not. These errors raise no exception and pass tests that compare outputs on a fixed set of inputs, because the error is not localized to any input. It is defined only relative to an intended equation that is not present in the program. Checking a program against the mathematics it is meant to implement is the aim of verification in scientific computing (Oberkamp and Roy, 2010).

Existing dynamic analyses address input selection. Concolic testing (Godefroid et al., 2005), property-based testing, and fuzzing generate inputs that exercise program paths. For numerical code the paths are few and the arithmetic between branches carries the meaning, so path coverage alone does not establish that the computed quantity is correct. scikit-verify addresses the other half. It extracts the expression the code computes and checks it against a specification.

2 Description

The library exposes three operations. First, `to_sympy(fn, args)` executes `fn` with tracing values that record operations as they occur, and returns the resulting SymPy expression,

the branch conditions taken during the run, and a record of any compiled call it could not trace. The target function is unmodified.

```
def weighted_rms(x, w):
    return np.sqrt(np.sum(w * x**2) / np.sum(w))

to_sympy(weighted_rms, x, w).formula
# sqrt(Sum(w[j]*x[j]**2, (j, 0, 2)) / Sum(w[j], (j, 0, 2)))
```

Listing 1: Extracting the formula a function computes.

Second, `check_formula(fn, args, spec)`, and the decorator `@specifies(spec)` that presents it as a test, compare the extracted expression to a formula the user supplies. The comparison is symbolic, so a match holds for all inputs of the traced shape rather than for the sampled inputs, and a mismatch returns both expressions and an input that distinguishes them. Results are reported at levels from exact symbolic equality to an undecided region, each recording the shape at which it was obtained.

```
y = sympy.IndexedBase("y")

@specifies((y[0] + 4*y[1] + 2*y[2] + 4*y[3] + y[4]) / 3)
def test_simpson_rule():
    return (lambda v: simpson(v)), (samples,)
```

Listing 2: A docstring formula checked as a test.

Third, a single run follows one path, so `explore(fn, args)` extends a check to the remaining branches. It negates each recorded branch condition, queries Z3 for an input satisfying the negation, and re-executes, until every branch is reached or proven infeasible. It reports full coverage only when Z3 establishes that the path conditions cover the domain. By default `@specifies` checks the specification on every branch, so a passing test cannot hide an unexercised branch. A related form, `@specifies.property`, checks a stated property, such as entries summing to one, when a closed form is unavailable.

3 Related software

Dynamic symbolic tools for Python exist but target a different output. DART (Godefroid et al., 2005) and its descendants generate inputs that maximize path coverage. CrossHair analyzes Python functions against boolean contracts and reports counterexamples. Property-based tools such as Hypothesis sample inputs and check user assertions. In each case the symbolic expression, if formed at all, is internal and discarded once inputs or a pass/fail result are produced. scikit-verify instead treats the extracted expression as the deliverable and checks it against external mathematics. Table 1 summarizes the distinction.

4 Implementation

Tracing follows the peer architecture of Bruni et al. (2011). The symbolic engine is a library in the target language and observes the program through its own operator dispatch. Each

	Output	Oracle	Domain
DART / PyExZ3	test inputs	none	general
CrossHair	counterexample	boolean contract	general
Hypothesis	counterexample	user assertion	general
scikit-verify	formula and verdict	a stated formula	numerical

Table 1: scikit-verify keeps and checks the symbolic expression that related tools discard.

traced value pairs a concrete NumPy value with a SymPy expression. NumPy operations are intercepted through the `__array_ufunc__` and `__array_function__` protocols. Data-dependent branches are recorded by overriding the truth test, which returns the concrete result and appends the symbolic condition to the current path.

The rule throughout is to return an exact expression or report a refusal. Compiled routines that cannot be traced, such as LAPACK solvers and eigendecompositions, become named symbols checked against a defining equation on the concrete arguments (Blum and Kannan, 1989). A solve is checked against $Ax = b$, and a singular value decomposition against $U \text{diag}(s)V^\top = A$. When a specification’s inputs flow through such a call, the result records it, as shown below. The solve is sealed and checked, while the formula-level check states that it cannot reach the inputs behind the call.

```
out = to_sympy(lambda A, b: np.linalg.solve(A, b).sum(), A, b)
out.formula      # Sum(solve_0[j], (j, 0, 1))
out.unchecked    # ('solve', {'square': 'ok', 'residual': 'ok'})
```

Listing 3: A compiled call sealed and checked against its defining equation.

5 Evaluation

The repository contains scripts that measure coverage by comparing traced values against untraced executions. On the public NumPy API, 293 of 310 functions trace exactly, and of those, 274 receive branch-coverage proofs, with the rest reporting a path limit or an undecidable region. Comparable boards cover scipy, scikit-learn, and statsmodels, and no function produced an expression that disagreed with the values the code returned. We also checked 33 formulas taken from the documentation of numpy, scipy, and scikit-learn functions. Of the 33, 31 matched at an exact or float-constant level, and the two that did not involved compiled calls, reported as such.

Applied to the test suite of an external Markov-chain package, the defining-equation check on its stationary-distribution routine reported a failure. The compiled backend passed a C-ordered array to a LAPACK routine expecting column-major order, producing incorrect distributions for transition matrices that are not doubly stochastic. The existing tests did not catch it because they used symmetric matrices, for which the incorrect and correct results coincide.

6 Limitations

The guarantees hold at the shape of the traced inputs. A property proven at length five is not thereby proven at length six, and generalizing across shapes is future work. Checking is over exact arithmetic, so a confirmed formula may still lose accuracy in floating point. We address whether the code computes the intended expression, not its numerical stability. Branch coverage rests on Z3, so guards outside its decidable theories, such as those involving transcendental functions, are reported as undecided rather than resolved. Finally, a user-supplied specification is only as correct as the user makes it. The tool checks the code against the stated formula, not the formula against the mathematics it was meant to capture.

7 Availability and Conclusion

scikit-verify is implemented in Python and depends only on NumPy, SymPy, and z3-solver, all open source. It is on PyPI as `scikit-verify` under the BSD-3-Clause license. The test suite has 1519 tests with 96% line coverage and runs under continuous integration. Two correctness oracles run within it, one that generates a formula and its code so the expected result is known, and a differential test of the solver translation against SymPy. Documentation on Read the Docs covers installation, tutorials, worked examples including a spline penalty matrix checked against its defining integral, design documents, and a full API reference. The project has 8 contributors, 27 stars, and 12 forks on GitHub.

An optional companion, installed as `scikit-verify[mcp]`, exposes the tracer to coding agents through the Model Context Protocol. An agent that writes numerical code can trace its own output, compare the resulting formula against a reference, and receive a deterministic verdict, instead of asking a model whether its own code looks correct. As more numerical code is written with model assistance, a plausible implementation can miss a mathematical assumption that comparing outputs never reveals.

Future work extends the checked-atom mechanism so that a sealed call’s defining equation serves as an assumption a solver can reason under, allowing facts to be proven through compiled boundaries rather than only up to them. The source is at <https://github.com/aadya940/scikit-verify>.

References

- Manuel Blum and Sampath Kannan. Designing programs that check their work. In *Proceedings of the ACM Symposium on Theory of Computing (STOC)*, pages 86–97, 1989.
- Alessandro Bruni, Tim Disney, and Cormac Flanagan. A peer architecture for lightweight symbolic execution. Technical report, University of California, Santa Cruz, 2011.
- Patrice Godefroid, Nils Klarlund, and Koushik Sen. DART: Directed automated random testing. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 213–223, 2005.
- William L. Oberkamp and Christopher J. Roy. *Verification and Validation in Scientific Computing*. Cambridge University Press, 2010.